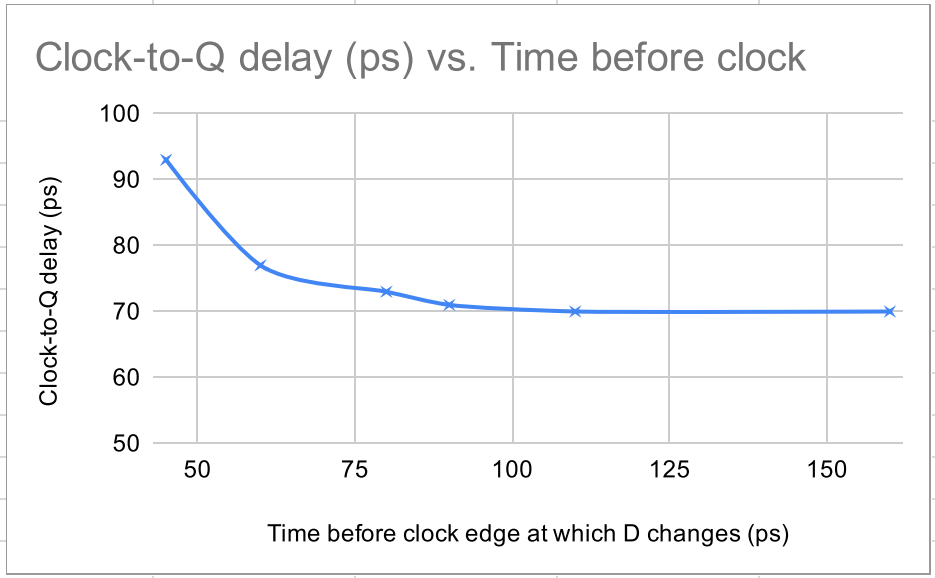
1A

Time before clock edge at which D changes (ps)	Clock-to-Q delay (ps)
160	70
110	70
90	71
80	73
60	77
45	93



Flat region starts at 70. Degradation rule on 70 (70*1.05) equals:

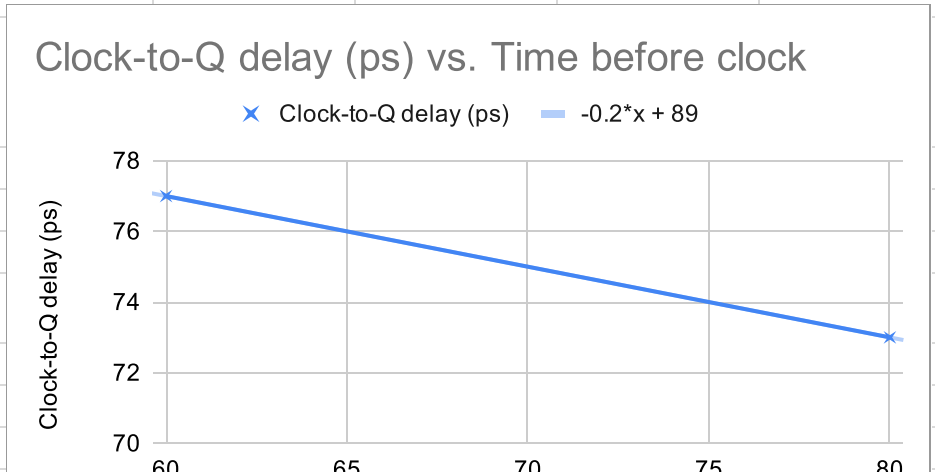
73.5

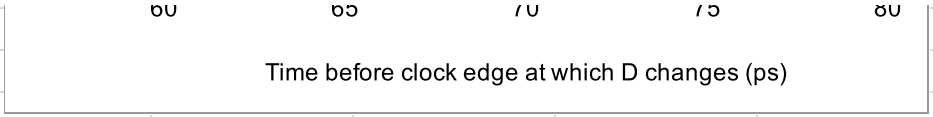
Value of 73.5 clock-to-q delay bounded by data points at 60ps time before clock edge (77 ps delay) and 80ps time before clock edge (73ps delay). Have to interpolate between the two to find where the delay value equals 73.5

Time before clock edge at which D changes (ps)	Clock-to-Q delay (ps)
80	73
60	77

At y=73.5, x equals

77.5 **picoseconds**





Assuming that register A is a master/slave register, as suggested by 1b, the values are computed via a feedback loop that fights for dominance by cutting off the other path. If the input gate closes too soon, the flipflop will not have all the information it needs, and will take longer to decide upon a value.

1B

Time after rising clock edge (ps)	Observed register behavior	
10	Incorrect value captured	Fail!
25	Output glitch observed	Fail!
40	Correct output, no glitch	Just barely correct!
60	Correct output, no glitch	Comfortably correct!

I would choose a 40ps hold time, as a conservative estimate, as it is the least amount of time observed to produce no glitch or incorrect value.

A hold time violation is different from a setup time violation as the setup time can be fixed by altering a clock period! Hold time violations happen when future data arrives too quickly and the following flipflop has too much data to hold - setup violations happen when the flipflop is trying to setup the next evaluation of data and the information doesn't arrive in time.

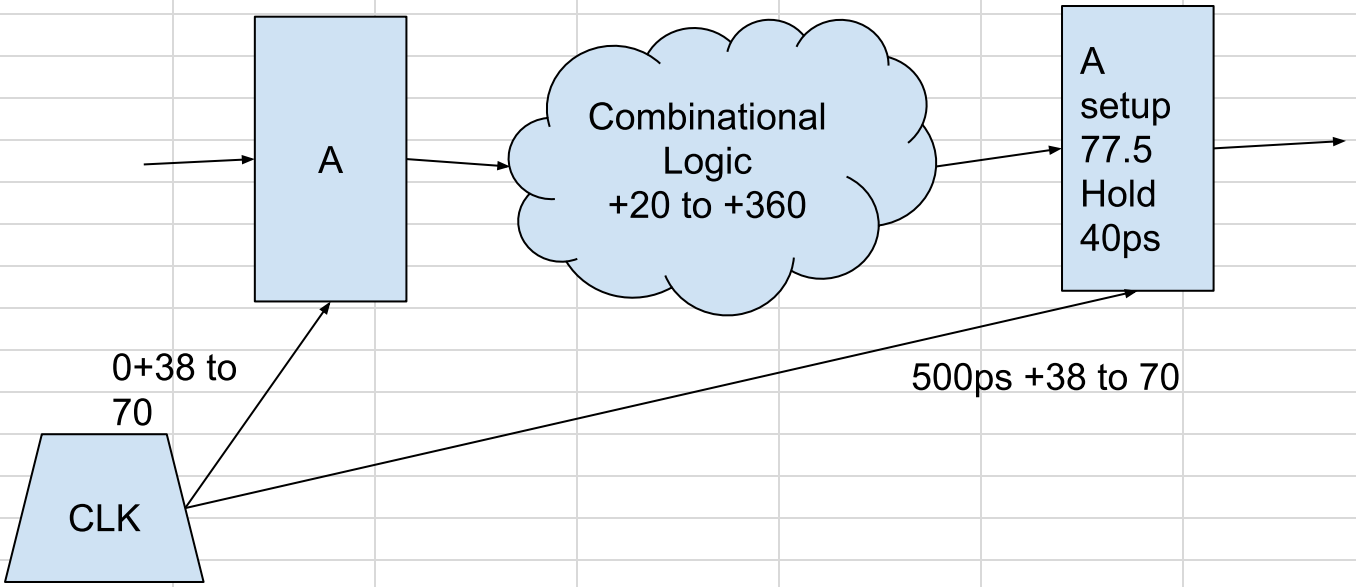
1C

Parameter	Register A	Register B
Maximum clock-to-Q delay ps	70	82
Minimum clock-to-Q delay ps	38	45
Setup time ps	77.5	50
Hold time ps	40	55

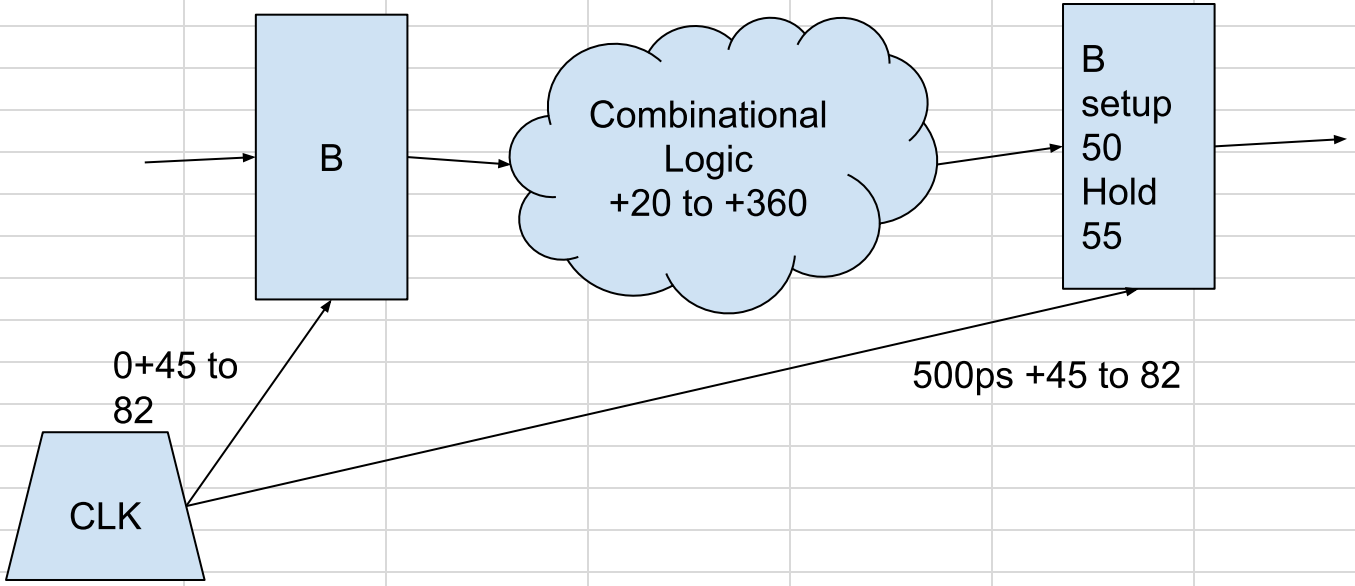
Note: values from part a, part b

Clock input capacitance fF	4	7

Two identical registers are connected by a combinational circuit with a maximum delay of 360 ps and a minimum delay of 20 ps. Assume a clock period of 500 ps, zero clock skew, and no clock uncertainty. For both register choices, determine the setup and hold timing margins. Which design meets both timing constraints?



setup:	minimum clock uncertainty +min clock period greater than clock to q max+comb logic max + setup max					
	500	greater than?	70+360+77.5			
	500	not greater than	507.5			
	setup violation					
hold:	minimum clock update delay (38) + combinational logic minimum (20) greater than hold time					
	58	greater than	40			
	no hold time violation!					



setup:	minimum clock uncertainty +min clock period greater than clock to q max+comb logic max + setup max						
	500	greater than?	82+360+50				
	500	greater than	492				
	no setup violation						
hold:	minimum clock update delay (45) + combinational logic minimum (20) greater than hold time (55)						
	65	greater than	55				
	no hold time violation!						
	Design B meets both timing constraints						

1D	Suppose the goal is to operate at the highest possible clock frequency while minimizing clock- distribution power. Which register would you select? Discuss the tradeoff between clock-to-Q delay, setup time, hold time, and clock input capacitance.

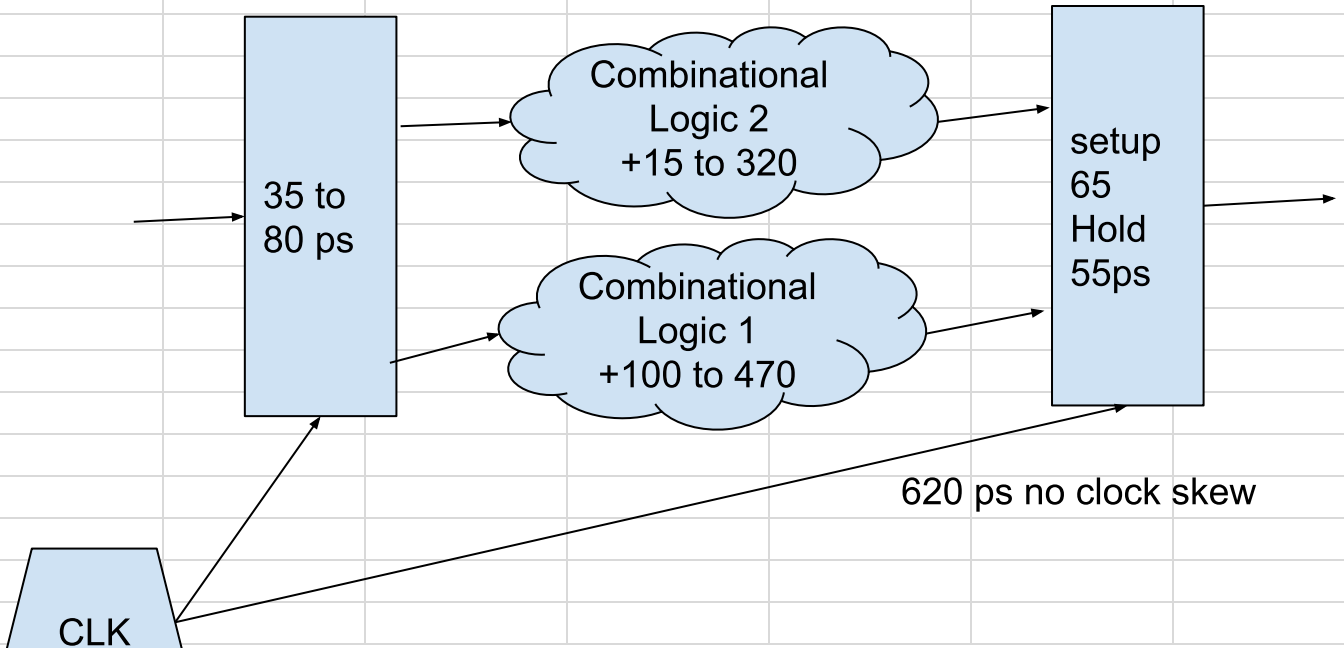
Parameter	Register A	Register B
Maximum clock-to-Q delay ps	70	82
Minimum clock-to-Q delay ps	38	45
Setup time ps	77.5	50
Hold time ps	40	55
Clock input capacitance fF	4	7

Register A's clock period, when adjusted to fix timing violations, is longer than Register B. Therefore, to simply get the highest possible clock frequency I would suggest Register B. There is a longer Clock To Q delay in B than A, which isn't ideal, and in addition to this Register B will use more power than register A. Therefore, I would suggest A over B due to the near doubling power requirements, evident from the capacitance, if and only if I can assume that the setup violations were corrected.

2A

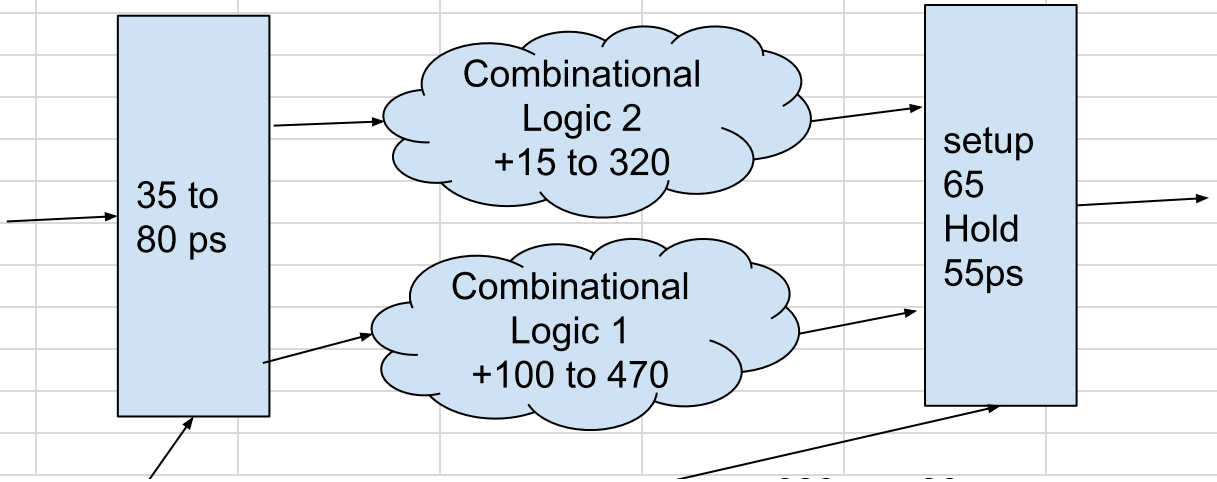
Parameter	Value
Maximum clock-to-Q delay of R1	80
Minimum clock-to-Q delay of R1	35
Setup time of R2	65
Hold time of R2	55
Setup clock uncertainty	15
Hold clock uncertainty	10
Clock Period	620

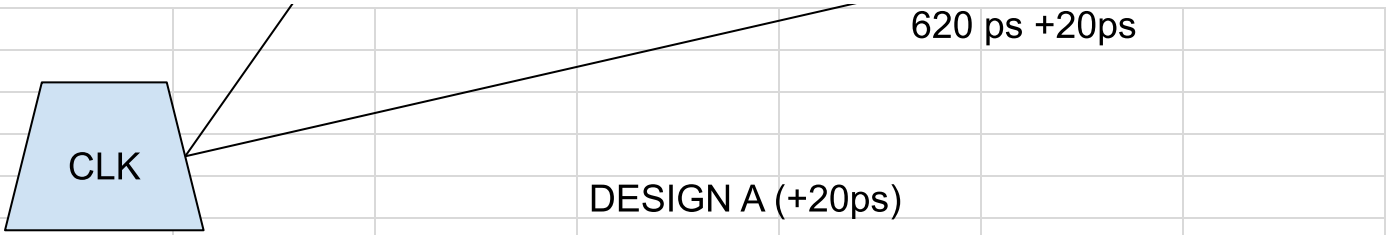
Path	Min Delay	Max Delay
1	100	470
2	15	320



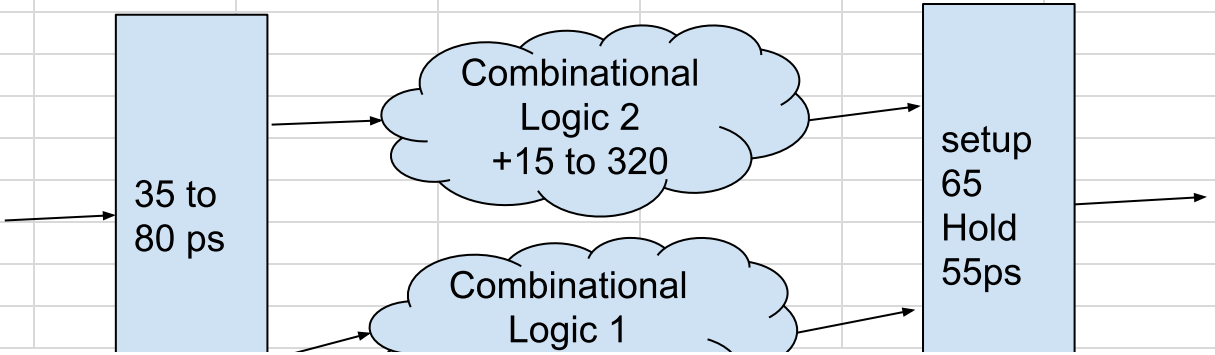
setup:	minimum clock uncertainty +min clock period greater than clock to q max+comb logic max + setup max + setup uncertainty				
	620	greater than	80+(320 OR 470) + 65 + 15		
	Case 1:	620	LESS THAN	630	setup violation!
	Case 2:	620	greater than	480	(no setup violation)
	Worst case is case 1, setup violation of 10ps.				
hold time:	minimum clock update delay 35 + combinational logic minimum (100 OR 15) greater than hold time (55) + hold uncertainty				
	Case 1:	135	greater than	65	(no hold time violation)
	Case 2:	50	LESS THAN	65	hold time violation!
	Worst case is case 2, hold violation of 15ps.				

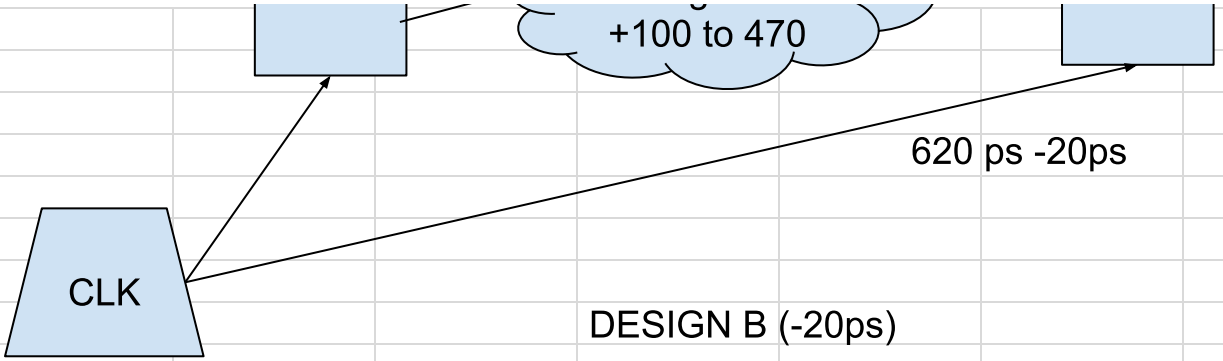
2B





setup:	minimum clock uncertainty +min clock period greater than clock to q max+comb logic max + setup max + setup uncertainty				
	620+20	greater than	80+(320 OR 470) + 65 + 15		
	Case 1:	640	greater than	630	no setup violation anymore!
	Case 2:	640	greater than	480	(no setup violation)
	Worst case is case 1, but no setup violation				
hold time:	minimum clock update delay 35 + combinational logic minimum (100 OR 15) greater than hold time (55) + hold uncertainty + clock skew				
	Case 1:	135	greater than	85	(no hold time violation)
	Case 2:	50	LESS THAN	85	WORSE hold time violation!
	Worst case is case 2, hold violation of 35ps.				





setup:	minimum clock uncertainty +min clock period greater than clock to q max+comb logic max + setup max + setup uncertainty				
	620-20	greater than	80+(320 OR 470) + 65 + 15		
	Case 1:	600	LESS THAN	630	worse setup violation!
	Case 2:	640	greater than	480	(no setup violation)
	Worst case is case 1, setup violation of 30				
hold time:	minimum clock update delay 35 + combinational logic minimum (100 OR 15) greater than hold time (55) + hold uncertainty + clock skew				
	Case 1:	135	greater than	45	(no hold time violation)
	Case 2:	50	greater than	45	(no hold time violation)
	Worst case is case 2, no hold violations				
Clock skew affects hold time and setup time differently because clock skew hurts hold time measurements but provides more time for setup time constraints.					
Clock skew 1 is better for improving setup time violations, and clock skew 2 is better for improving hold time violations.					

2C

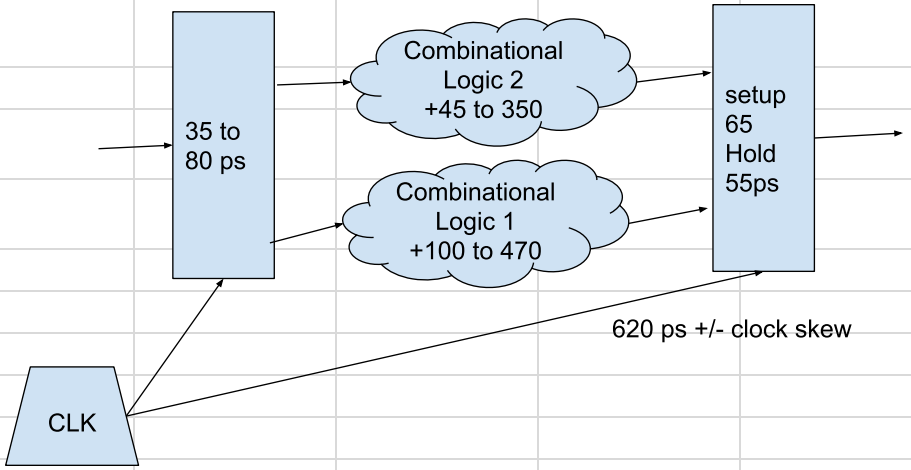
Off the top of my head, I don't believe it's possible; a clock skew that would benefit one would harm the other. This is because clock skews affect hold time and setup time in opposite ways, and to mend one kind i'd need a positive skew and to mend the other I'd need a negative skew.

2D

NEW VALUES

Parameter	Value
Maximum clock-to-Q delay of R1	80
Minimum clock-to-Q delay of R1	35
Setup time of R2	65
Hold time of R2	55
Setup clock uncertainty	15
Hold clock uncertainty	10
Clock Period	620

Path	Min Delay	Max Delay
1	100	470
2	45	350



setup	clock skew +min clock period greater than clock to q max+comb logic max + setup max + setup uncertainty			
	skew + 620	vs	80+470+65 +15	
Case 1:	620+skew	greater than	630	(skew >10)
Case 2:	620+skew	greater than	510	(no setup violation)

hold time:	minimum clock update delay 35 + combinational logic minimum (100 OR 45) greater than hold time (55) + hold uncertainty + clock skew					
	Case 1:	135	greater than	65		
	Case 2:	50	greater than	55+10-skew	skew less than 15	
choosing skew =10						
setup	clock skew +min clock period greater than clock to q max+comb logic max + setup max + setup uncertainty					
Case 1	630	Equal To	630			
Case 2	630	Greater than	510			
Hold time:	minimum clock update delay 35 + combinational logic minimum (100 OR 45) greater than hold time (55) + hold uncertainty + clock skew					
Case 1:	135	greater than	55+10-10			
Case 2:	80	greater than	55+10-10			
The Timing Repair is successful because it bridges the gap between setup time and hold time by working in conjunction with the buffer in the combinational logic.						
If P2 was combinationally longer than P1, this solution would not work I think. It depends on a precarious balance between the two, so changing the values of the actual combinational circuit blocks would also change the answer? perhaps I misunderstand the question.						

3A

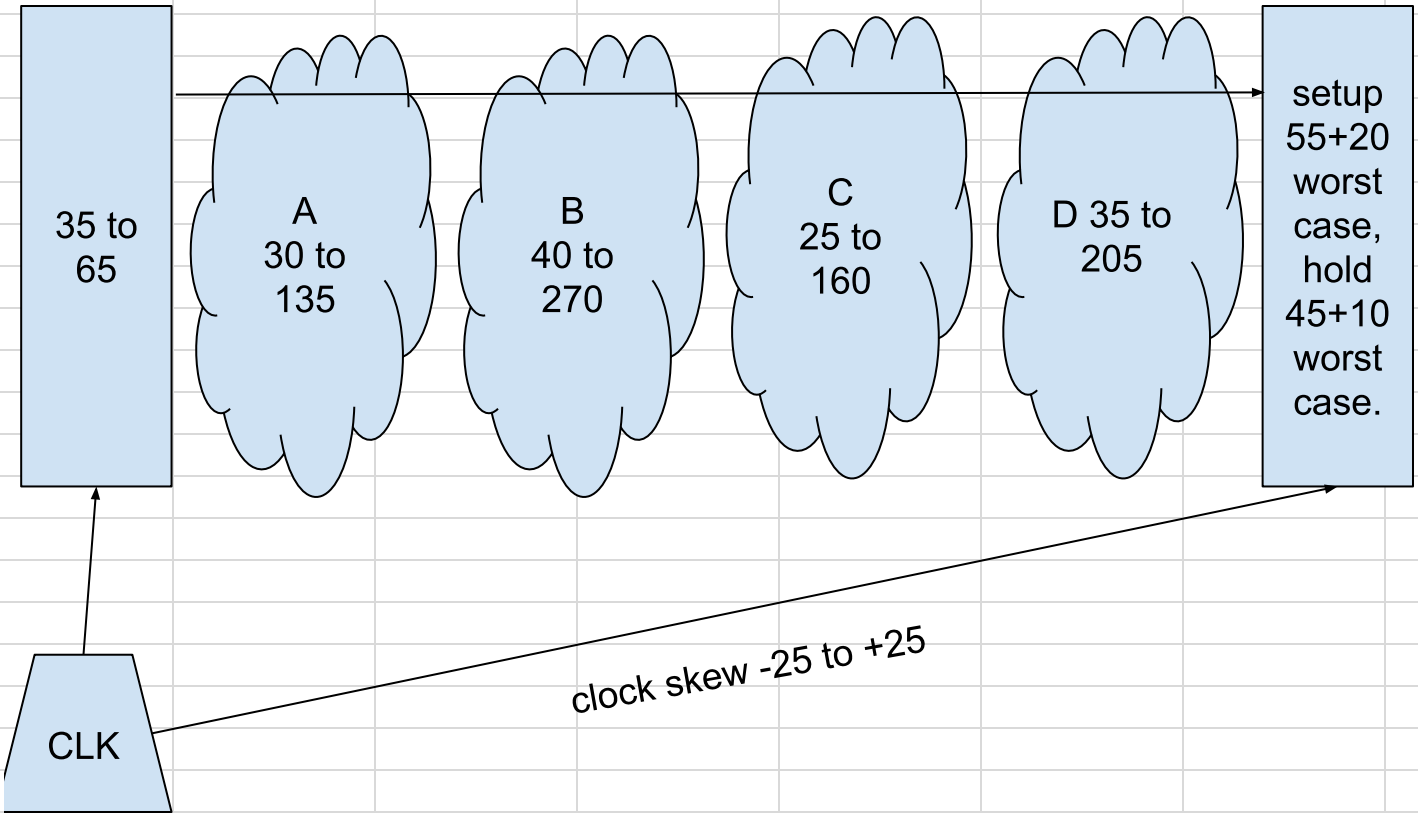
Logic Block	Min Delay	Max Delay
A	30	135
B	40	270
C	25	160
D	35	205

Parameter	Value
Max C to Q	65
Min C to Q	35
Setup	55
Hold	45

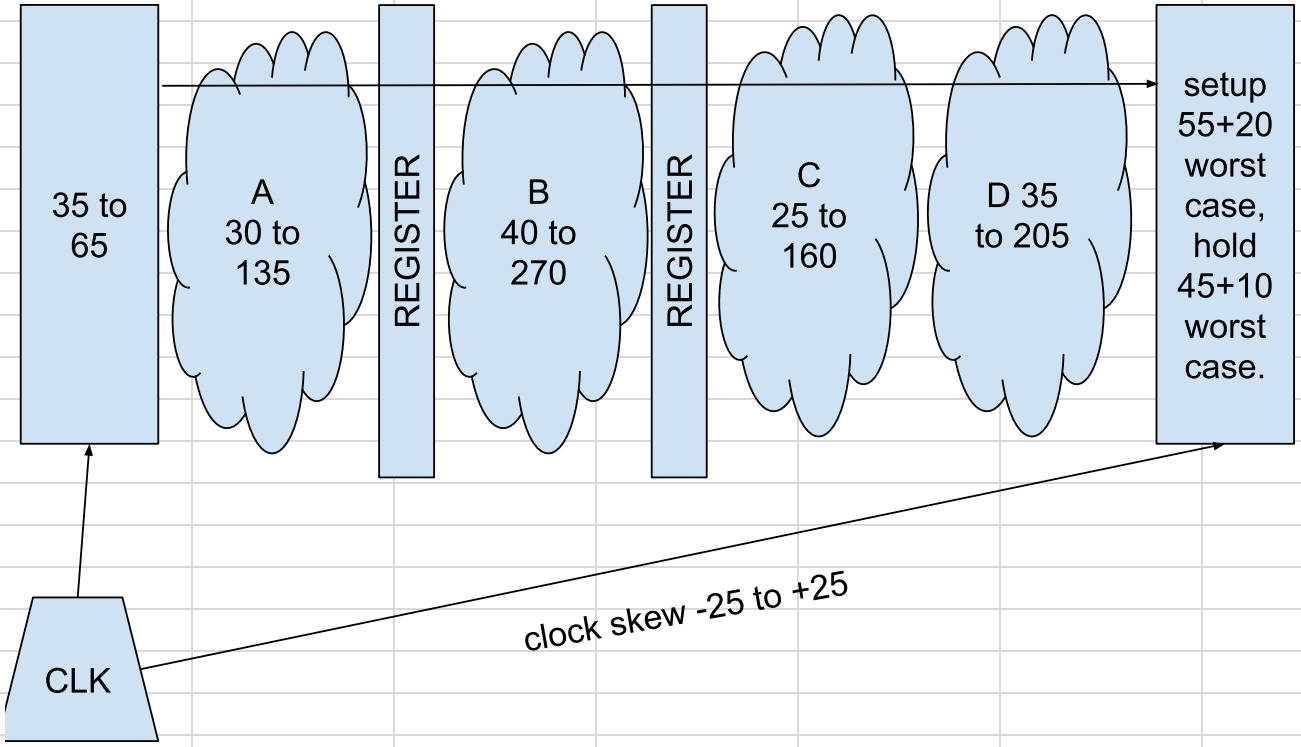
Clock skew between -25 to 25

Worst case setup uncertainty of 20

Worst Case Hold Uncertainty of 10



	Minimum clock period:		65+135+270+160+205+55+20+25			equals	935	
							(no pipelining)	
3B	160+205=	365						
	135+270=	405						
	Registers should be between A+B, and between B+C.							
	This is because B has the largest maximum time, and I don't see a better answer.							



	Longest combinational logic delay:			160 + 205 = 365				

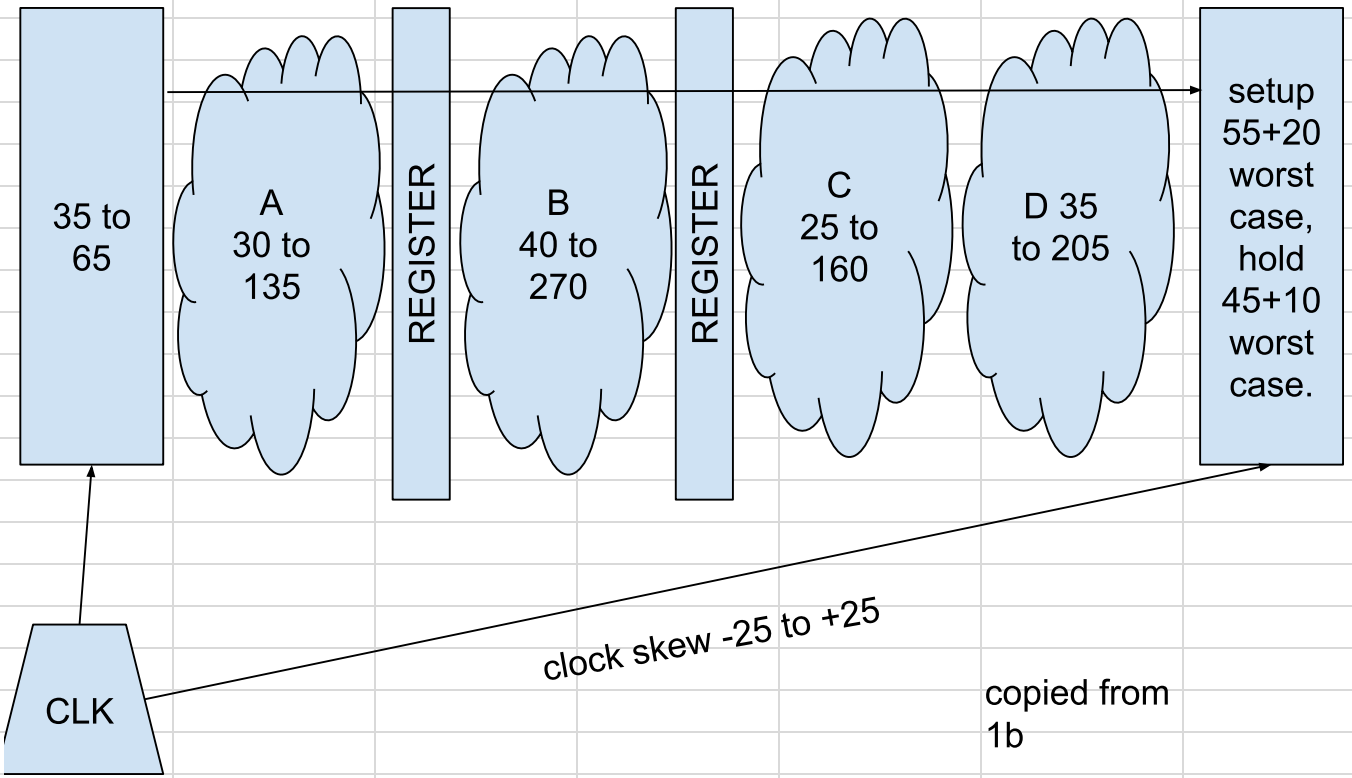
	Minimum Clock Period		65+365+setup+setup uncertainty+skew					
			530					
	Maximum Operating Frequency		equals 1/period					
			0.001886792453					
			or					
			1.9GHZ					
3C	Logic Block	Max Delay		Parameter	Value			
	A	135		Max C to Q	65			
	B	270		Min C to Q	35			
	C	160		Setup	55			
	D	205		Hold	45			
	Clock skew between -25 to 25							
	Worst case setup uncertainty of 20							
	Worst Case Hold Uncertainty of 10							
	ABC	**565**		D	205			
	AB	**405**		CD	365			
	A	135		BCD	**635**			
	Placing the register between B and C is optimal. the largest combinational segment in that orientation would be 405.							
	Minimum Clock Period		65+405+setup+setup uncertainty+skew					
		equals	570					
	frequency	equals	0.001754385965	or	1.8GHZ			
	In terms of operating frequency: the more dividing registers, the higher (and better) the Maximum Operating frequency.							

For steady state throughput, all have steady state throughput after the pipeline fills - the pipeline takes longer to fill, though, so it would depend on how many calculations and pieces of data one wanted to pass through.

Number of clock cycles: the number needed increases as one adds dividing registers.

Increasing the number of pipeline stages doesn't necessarily reduce the latency of an individual computation because dividing up the computations into separate blocks may remove inter-block efficiencies and value-passing.

3D

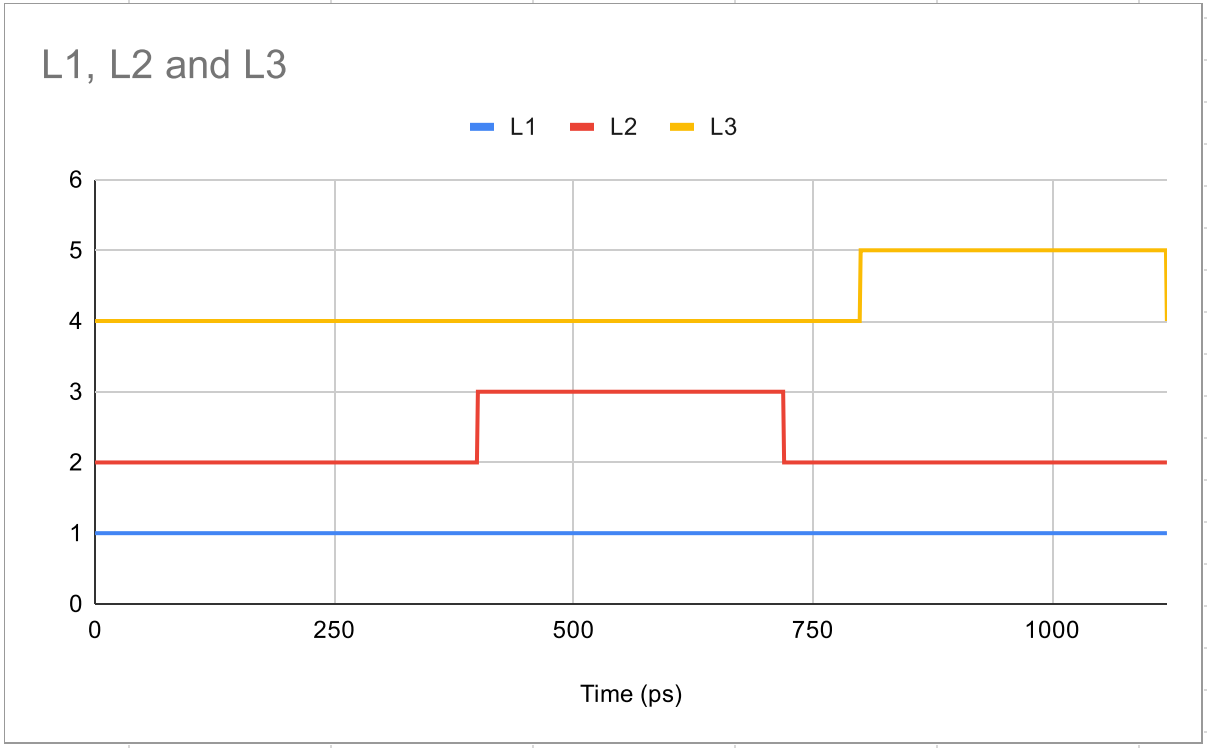


A	Hold time:	minimum clock update delay + combinational logic minimum greater than hold time + hold uncertainty + clock skew

30+(-25)	Greater than?	45+10+25					
5	Less Than	80	(minimum extra delay of 75)				
B	Hold time:	minimum clock update delay + combinational logic minimum greater than hold time + hold uncertainty + clock skew					
40-25	Greater than?	45+10+25					
15	less than	80	(minimum extra delay of 65)				
CD	Hold time:	minimum clock update delay + combinational logic minimum greater than hold time + hold uncertainty + clock skew					
60-25	Greater than?	80					
35	less than	80	minimum extra delay of 45				
These hold-time repairs would increase the minimum clock period that I calculated in part B, as adding 75 +65 +45 delays would require increasing the clock period by 75 to account for the largest delay required per clock signal.							

4A

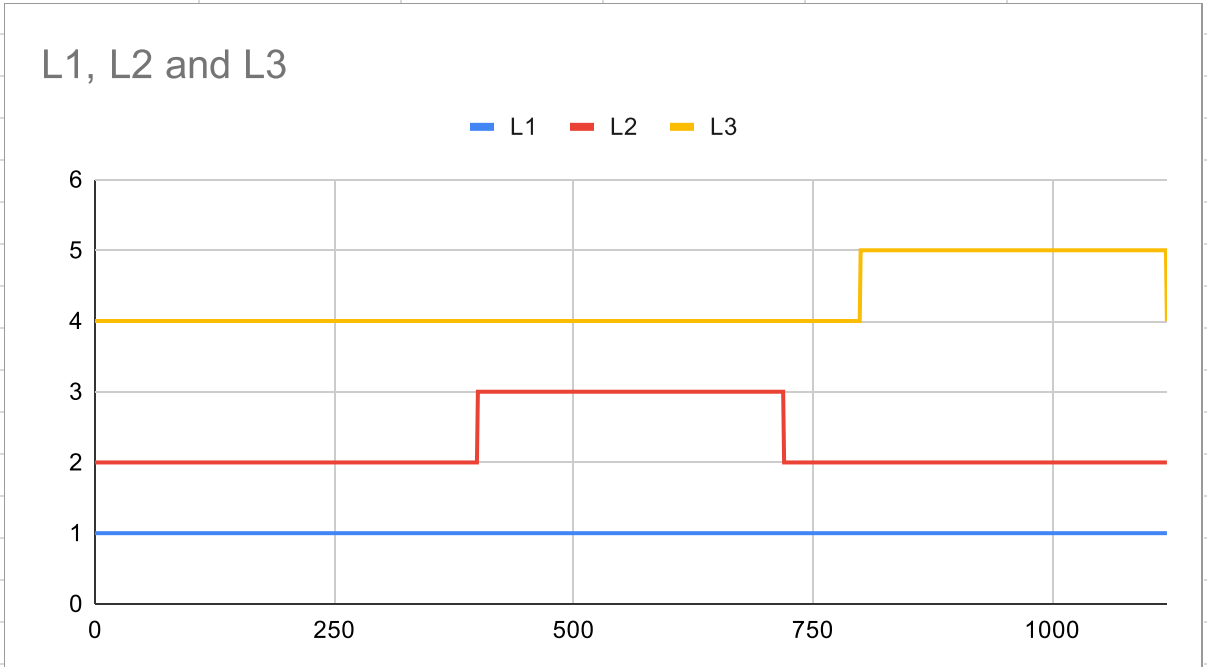
Time (ps)	L1	L2	L3
0	1	2	4
399	1	2	4
400	1	3	4
719	1	3	4
720	1	2	4
799	1	2	4
800	1	2	5
1119	1	2	5
1120	1	2	4



	L1 output at 65ps						
	L1 L2 delay of 480ps						
	L2 L3 delay of 420 ps						
	Data reached L2:		545ps				
	Released at 65	65					
	Delay of 480	545					
	Output of L2:		600ps		When on, data changes passed along as soon as possible.		
	Arrived at input at 545		545				
	Stable input of 55		600				
	Input of L3:		1020ps	(not good!!)			
	Output of L2 at	600					
	Delay of 420	1020					
4B							
	The data does not satisfy the set-up time requirement of L3 (L2 is fine).						
	L2:						
	arrives at 600, L2 off at 750; more than enough time.						
	(95ps leeway)						
	L3:						
	Arrives at 1020, L3 off at 1020, certainly not enough time to pass value forwards.						
	(50ps short)						
	The data would need to borrow 50ps of L2's time, starting from 670 (720-50)						

4C

L3 input at:	
965	(1020-55)
L2 output at:	
545	965-420
L2 input at:	
490	545-55
L1 output at:	
10	490-480
(no worry about L1 input)	
Therefore, delay of 10ps is possible.	

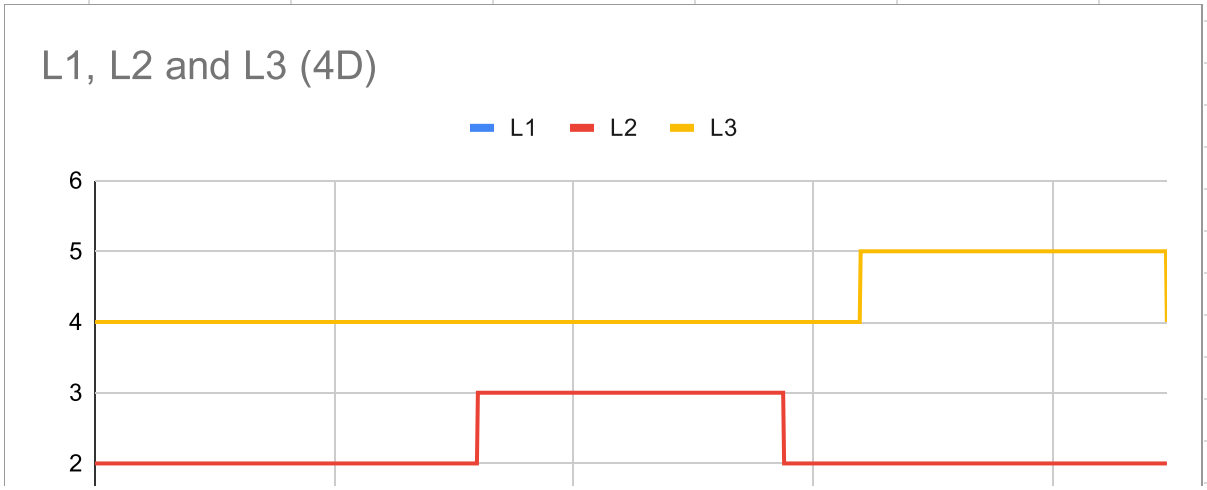


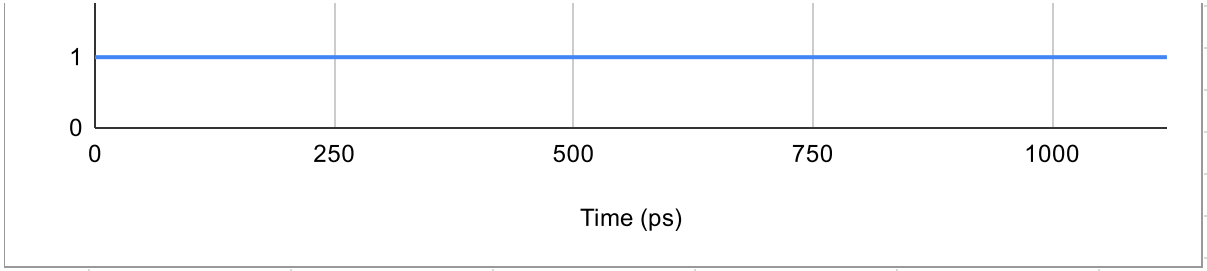
Time (ps)

If the L1 to L2 combinational delay increases by 60:
L1 will pass to L2 on time at 605 (setup time fine, hold time fine).
L2 will pass to L3 at 1080ps which is far too late by setup time And hold time.

4D

Time (ps)	L1	L2	L3
0	1	2	4
399	1	2	4
400	1	3	4
719	1	3	4
720	1	2	4
799	1	2	4
800	1	2	5
1119	1	2	5
1120	1	2	4





Original Logic Delays:

The launched data appears at the output of L1 65 ps after L1 turns ON.

The combinational logic between L1 and L2 has a maximum delay of 480 ps.

The combinational logic between L2 and L3 has a maximum delay of 420 ps.

L2 has a data-to-Q delay of 60 ps while it is ON. Both L2 and L3 require their input data to be stable at least 55 ps before the end of their ON intervals.

0: L1 on

65: L1 outputs value

400: register starts reading. Not too good! data arrives too late. should have arrived by 400-55.

545 : Data arrives at L2

605: L2 Output data. Late. Wouldn't output correct data.

....

The original logic delays would not satisfy the new timing requirements.

Latch-based timing allows for more flexible operations, time-wise, than registers. Registers are a hard open-close boundary, but latches can be triggered early or late and can change value if the value changes while the latch is transparent.

Latches are more complicated, because they are so flexible input-output wise, and so to model them is more complicated as well.

5A

Parameter	Value
Transparent data-to-Q delay L1	35 ps
Transparent data-to-Q delay L2	40 ps
Minimum combinational logic delay	60 ps
Maximum combinational logic delay	190 ps
Setup time of L2	30 ps
Hold time of L2	25 ps

Earliest:

115 (20+35+60)

Latest:

245 (20+25+190)

For minimum delay path, where data reaches L2 at 115:

115+40 = 155 ps

This would allow the data to pass through two latches in one clock-period - faster than expected. This is why it's called 'race through' - the data races through the circuit, too fast, and causes problems down the line.

For maximum delay path, where data reaches L2 at 245:

245+40 = 285

hold-window for L2 is between 240 and 240+25 (so, 265) and so the input of L2 can change within its hold timing window.

5B							
	In 5a we know that the data can't reach L2 between 240 and 265.						
	Minimum delay path: 115ps						
	Additional delay required: 265-115	150	ps				
	The earliest arrival at L2 will be at 115+150 (265ps)						
	The latest arrival will be 245+150 ps	395	ps				
	This is well in time to be caught by the 600-840 clock pulse, but it results in wasted time that could be latched (time-stolen) out.						
5C							
	L1 passes data at 20						
	115	arrives at L2					
	L1 turns opaque at 240		60ps of stable data				
	L2 setup opens at 300-30=270 (well in time)						
	L2 Hold time of 25 ps, until 325ps.						
	plus 40ps for processing (365)						
	The new clocking scheme prevents race through between L1 and L2 because there is a no-change zone for 60ps between one latch closing and the other opening.						
	It is also stable at 365ps, which is way before when it has to be stable at540ps.						
5D							

	Same Phase Latches are not good at all for preventing race through. They are good at minimum constraints, and allowing for time borrowing, but timing robustness is tough. Buffers are needed, which increases the clock period, but they are very finicky beasts. As a reward, though, time borrowing is allowed and in some senses encouraged!
	Two Phase Latches, where the latch closes, pauses, and then another opens, prevent race through all together. they are more complex, clock wise, but are very difficult to break. Time borrowing is allowed.
	Pos-edge triggered resistors are in the middle of the two options above, I believe. they are slightly vulnerable to clock skew and are easier to write. However, there is absolutely zero possibility to time-borrow!
	Overall, I'd suggest the two phase latches whilst prioritising reducing the amount of time spend in the unchanging value period between when one latch closes and the other latch opens. I feel like it permits the most variety in timing, whilst being really tough to misuse.
AI statement	
	I did not use ai to answer questions, but as suggested by Prof Joshi I checked my understanding of the topics with a general overview using AI when my notes weren't helpful.